



AI VIET NAM – AI Conquer 2026

Distance Metrics and Similarity Design for Recommendation Systems

Group Numby

Name	Position	Role
Le Quang Thanh	AI Engineer (Pipeline)	Team Leader
Vo Ngoc Gia Bao	QA/Reviewer	Member
Nguyen Van Vi	AI Engineer (Model)	Member
Le Thanh Lam	Tech Lead	Member
Pham Minh Dang Tran	AI Engineer (Data)	Member

I. Research Summary

This research addresses the crop recommendation problem on structured soil and weather data, where the goal is to recommend suitable crops for a given environmental condition. Rather than treating the task as a single-label classification problem, this work formulates it as a Top- K ranking problem so that multiple plausible crops can be returned for each query.

A limitation of many previous crop recommendation methods is that they focus mainly on top-1 classification accuracy and do not analyze recommendation ranking quality or retrieval efficiency in detail. This motivates a retrieval-based framework that can evaluate both recommendation quality and inference cost.

The proposed method consists of two distance-based recommenders: an exact class-wise KNN baseline and a K-Means++ accelerated KNN variant. The pipeline includes dataset loading, stratified train/validation/test splitting, median imputation, feature standardization, model tuning, and ranking-based evaluation. The models are evaluated using MAP@ K , NDCG@ K , HitRate@ K , latency per query, and candidate reduction ratio.

Experiments are conducted on the Crop Recommendation dataset with seven input features: nitrogen, phosphorus, potassium, temperature, humidity, pH, and rainfall. On the test set, the exact KNN baseline reaches test MAP@5 between 0.974 and 0.992 and NDCG@5 between 0.981 and 0.994 depending on the distance metric, with HitRate@5 = 1.000 throughout and per-query latency of 0.06–0.12 ms against the full 1,760-sample candidate pool. The K-Means++ accelerated model matches or slightly exceeds the baseline on every metric (e.g., Manhattan: MAP@5 = 0.9920, NDCG@5 = 0.9941, tied with the baseline) while reducing the average number

of examined candidates by 79–87%. However, its per-query latency is $6\text{--}23\times$ higher (0.82–1.47 ms), showing that on a dataset of this size the candidate-pruning benefit does not yet translate into a wall-clock speed-up under the current Python implementation.

Overall, this research provides a reproducible Top- K crop recommendation framework that goes beyond standard classification evaluation and explicitly studies the trade-off between ranking quality and retrieval efficiency. The results show that the dataset is highly learnable with distance-based methods, but also reveal that approximate search can introduce implementation overhead that limits practical speed gains on a small benchmark.

Contents

I.	Research Summary	1
II.	Introduction and Research Questions	6
II.1.	Research Problem and Hypothesis	7
III.	Related Work	8
III.1.	Traditional and Distance-Based Methods	8
III.2.	Classical Machine Learning Models	8
III.3.	Deep Learning Approaches	9
III.4.	Pretrained Models and Foundation Models	9
III.5.	Studies on Similar Datasets and Benchmarks	9
III.6.	Research Gap and Motivation	9
IV.	Methodology and Research Architecture	10
IV.1.	Problem Formulation	10
IV.2.	Baseline Method	10
IV.3.	Main Method	11
IV.4.	Data Processing Pipeline	11
IV.5.	Evaluation Strategy	11
IV.6.	Why This Method Works for the Problem	12
IV.7.	Summary of the Proposed Pipeline	12
V.	Data and Experimental Design	12
V.1.	Dataset	13
V.2.	Data Splitting	14
V.3.	Data Preprocessing	16
V.4.	Principal Component Analysis	16
V.5.	Model Configurations	17
V.6.	Distance Metrics	20
V.7.	Training and Implementation Setup	20
V.8.	Evaluation Setup	21
V.9.	Model Selection and Experimental Protocol	23

VI.	Results and Discussion	24
VI.1.	Test-Set Results: Baseline vs. Main Method	25
VI.2.	Quantitative Comparison Between Baseline and Hybrid	25
VI.3.	Per-Crop Performance Analysis	26
VI.4.	Summary of Results	27
VI.5.	Effect of Distance Metric and Neighbors per Crop on the Baseline	28
VI.6.	Effect of K-Means++ Index Granularity on the Hybrid Model	29
VI.7.	Classification-View Ablation: Effect of K and of the K-Means Index (Notebook Appendix A)	30
VI.8.	Latency Threshold and Component-Level Profiling	31
VI.9.	Weak Multi-Label Ground Truth for Domain-Informed Evaluation	33
VI.10.	Bootstrap Confidence Intervals: Single-Label vs. Weak Multi-Label Ranking Quality	33
VI.11.	Summary of Results and Ablation Study	34
VI.12.	Discussion	35
VII.	Conclusion and Future Work	36
VII.1.	Conclusion	36
VII.2.	Future Work	36
VIII.	References	38
	Appendix	39

List of Tables

1	Summary of the cleaned Crop Recommendation Dataset.	14
2	Example observation from the Crop Recommendation Dataset.	14
3	Explained variance obtained from PCA on the training set.	17
4	Selected KNN baseline parameters based on validation performance.	18
5	Selected KNN + K-Means++ parameters based on validation performance.	19
6	Common experimental settings.	21
7	Final test-set results for the KNN baseline and the KNN + K-Means++ hybrid with TOP_K = 5.	25
8	Effect of the K-Means++ search index relative to the exact KNN baseline on the test set.	25
9	Crops with the lowest per-class NDCG@5, best test configuration (KNN + K-Means++, Manhattan, 20 test samples per crop).	27
10	Validation MAP@5 / NDCG@5 of the KNN baseline as a function of <i>neighbors_per_crop</i> , by distance metric.	28

11	Representative validation configurations for the KNN + K-Means++ hybrid, showing the effect of $n_clusters$ on the candidate ratio at near-optimal MAP@5.	29
12	KNN vs. KNN++ (K-Means-indexed KNN) classification accuracy and predict time, $k = 5$ (notebook Appendix A).	31
13	Latency benchmark against the 1.0 ms p95 service-level objective, by model and distance metric (100-query workload, 3 warm-up + 10 measured runs).	32
14	Bootstrap 95% CI for MAP@5 and NDCG@5 under the weak multi-label ground truth (2,000 resamples).	33
15	Paired bootstrap delta (KNN + K-Means++ minus Exact KNN) under the weak multi-label ground truth.	34

List of Figures

1	System Objective Overview: Charting the Course for Smart Farming Excellence.[1]	6
2	Overall methodology pipeline of the crop recommendation system.	12
3	Workflow of the proposed K-Means++ accelerated KNN retrieval method.	18
4	Quality-search trade-off: MAP@5 vs. average candidate ratio for the baseline (candidate ratio = 1) and the K-Means++ hybrid (candidate ratio < 1), by distance metric.	26
5	Per-crop NDCG@5 for all 22 crop classes, best test configuration (KNN + K-Means++, Manhattan).	27
6	From-scratch KNN classification accuracy as a function of k , for the three distance metrics (notebook Appendix A).	30
7	Median (p50) and p95 per-query latency by model and distance metric, against the 1.0 ms service-level threshold (dashed line).	32

II. Introduction and Research Questions

Crop recommendation is an important decision-support problem in precision agriculture. Given soil nutrient measurements and environmental conditions, the objective is to recommend crops that are most suitable for a particular field. In this work, the problem is formulated as a Top- K retrieval task rather than a conventional single-label classification problem, allowing the system to return multiple candidate crops ranked by similarity.

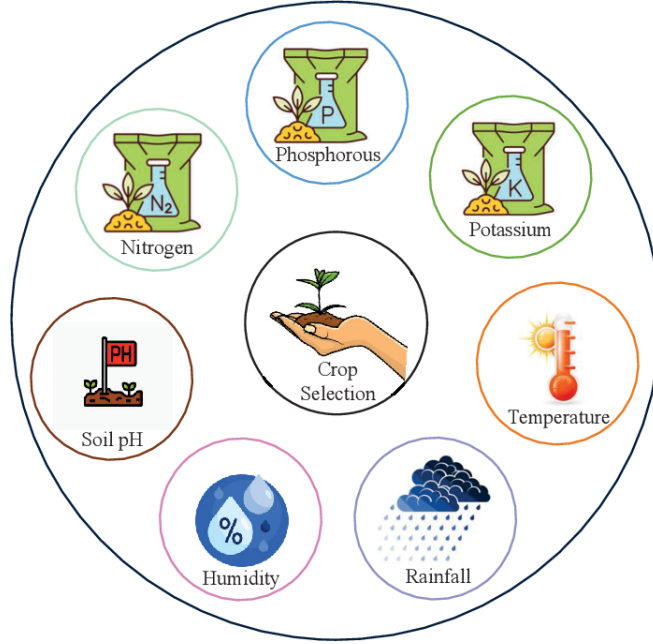


Figure 1: System Objective Overview: Charting the Course for Smart Farming Excellence.[1]

This problem is important both academically and practically. From an application perspective, crop recommendation can support farmers and agricultural planners in making data-driven planting decisions. From a research perspective, the task is challenging because crop suitability depends on nonlinear interactions among multiple numerical features, and several crops may be valid under similar conditions. As a result, methods that only optimize top-1 classification accuracy do not fully capture the behavior needed in a recommendation setting.

A key limitation of many existing approaches is that they focus on predicting a single crop label and evaluate performance mainly with classification metrics. Such methods do not explicitly study ranking quality, candidate pruning, or the trade-off between recommendation effectiveness and inference efficiency. In addition, because the data are structured and relatively low-dimensional, the choice of distance metric, preprocessing strategy, and search method can have a substantial impact on performance.

To address these issues, this study aims to build a reproducible crop recommendation framework that compares an exact retrieval baseline with an accelerated search strategy. The work first establishes a class-wise exact KNN baseline, then proposes a K-Means++ accelerated KNN variant to reduce the search space while preserving ranking quality. The two methods are evalu-

ated using ranking-oriented metrics such as MAP@K, NDCG@K, and HitRate@K, together with latency and candidate reduction measurements.

The main contributions of this research are summarized as follows:

- We build a clear baseline for the crop recommendation problem using an exact class-wise KNN retrieval method.
- We propose and evaluate a K-Means++ accelerated KNN strategy to improve search efficiency while maintaining recommendation quality.
- We conduct experiments on the Crop Recommendation dataset and compare the methods using ranking metrics and inference-efficiency metrics.
- We analyze the results, limitations, and error patterns to better understand the behavior of distance-based crop recommendation models.

II.1. Research Problem and Hypothesis

This research addresses the **crop recommendation** problem, formulated as a **Top- K ranking and retrieval task** on structured tabular data. Given a set of soil and weather attributes, the objective is to return a ranked list of the most suitable crop classes for a specific agricultural condition.

- **Problem name:** Crop recommendation / Top- K crop retrieval.
- **Input:** A tabular feature vector containing seven numerical variables: nitrogen, phosphorus, potassium, temperature, humidity, pH, and rainfall.
- **Output:** A ranked list of the top- K recommended crop labels, together with similarity-based scores and diagnostic information such as candidate count and search efficiency.
- **Research objective:** The goal is to improve the quality and practicality of crop recommendation by balancing ranking performance and inference efficiency. In particular, the study aims to evaluate whether an accelerated retrieval strategy can preserve recommendation quality while reducing the search space.
- **Evaluation metrics:** MAP@K, NDCG@K, HitRate@K, Precision@K, Recall@K, mean rank when hit, latency per query, and candidate reduction ratio.

The main research hypothesis is that a distance-based recommendation framework can produce strong Top- K crop rankings on this dataset, and that a K-Means++ accelerated search strategy can reduce the number of candidates examined while maintaining comparable ranking quality.

Hypothesis:

Using a retrieval-based Top- K recommender with proper preprocessing and clustering-based candidate pruning can improve efficiency without significantly degrading recom-

mentation quality on the crop recommendation dataset.

Even though the study does not introduce a completely new AI architecture, it still defines a clear research problem by comparing an exact KNN baseline with an accelerated K-Means++ search variant, then analyzing their behavior on a fixed agricultural benchmark.

III. Related Work

Crop recommendation aims to identify the most suitable crop based on soil nutrient and environmental conditions. Common input variables include soil nutrient measurements (e.g., nitrogen, phosphorus, and potassium) together with environmental factors such as temperature, humidity, pH, and rainfall, while the output is a recommended crop for a given agricultural condition [2].

Previous studies generally categorize crop recommendation systems into two broad groups: rule-based systems and artificial intelligence (AI)-based systems [3]. Rule-based approaches rely on predefined agricultural knowledge and decision rules, whereas AI-based approaches employ data-driven techniques, including machine learning and deep learning, to model the relationship between soil, climate, and crop suitability.

III.1. Traditional and Distance-Based Methods

Early crop recommendation systems primarily relied on expert knowledge and rule-based decision making, where predefined agricultural rules and threshold-based logic were used to recommend suitable crops under specific environmental conditions [3].

Among similarity-based approaches, the k-nearest neighbors (kNN) algorithm remains a simple yet effective baseline because it retrieves the most similar labeled instances according to a predefined distance metric [4]. In tabular crop recommendation, kNN is attractive because it requires little model complexity, supports an instance-based learning algorithm that makes predictions from nearby training samples, and can be adapted to different distance metrics. However, its prediction cost grows with the size of the training set, and performance depends heavily on feature scaling, metric choice, and the number of neighbors considered.

III.2. Classical Machine Learning Models

A large body of work on crop recommendation uses classical supervised learning models such as Decision Trees, Random Forests, SVMs, and Gradient-boosted methods [5]. These models are popular because they handle structured tabular data well and often achieve strong accuracy with relatively limited feature engineering [5]. In particular, tree-based ensemble methods such as Random Forest are effective at capturing nonlinear interactions between soil and weather variables while also providing interpretable feature importance scores [5], [6].

Despite their effectiveness, many classical models are optimized for single-label classification rather than ranked recommendation. As a result, they typically output only one predicted crop class, even though multiple crops may be plausible for a given condition. This makes them less suitable when the application requires a ranked Top-K list, confidence ordering, or decision

support under uncertainty.

III.3. Deep Learning Approaches

Deep learning models, particularly neural networks and multilayer perceptrons (MLPs), have also been explored for crop recommendation and have reported competitive classification accuracy on benchmark datasets [5].

However, most existing studies formulate crop recommendation as a classification problem rather than a retrieval task. Consequently, they provide limited insight into the impact of different distance metrics on Top-K recommendation quality.

III.4. Pretrained Models and Foundation Models

Compared with vision or language tasks, pretrained models and foundation models are not yet dominant in crop recommendation, mainly because the input data are structured tabular measurements rather than unstructured text or images [7]. Most existing crop recommendation studies train models directly on task-specific agricultural datasets rather than fine-tuning large pretrained tabular models.

Recent foundation-model research for tabular data suggests that pretrained representations may become more relevant in the future, especially when data are scarce or heterogeneous.

III.5. Studies on Similar Datasets and Benchmarks

A widely used benchmark in crop recommendation research is the Crop Recommendation Dataset, which contains seven soil nutrient and environmental features together with crop labels. Many studies use this dataset to compare different machine learning models under the same experimental setting and report their classification performance [8], [9].

In practice, a recommendation system should often return more than one candidate crop, especially when multiple classes are close in feature space. For that reason, evaluation should consider ranking-aware metrics such as MAP@K, NDCG@K, and HitRate@K, as well as operational metrics such as inference latency and candidate reduction. These aspects are rarely analyzed in depth in the existing literature.

III.6. Research Gap and Motivation

Based on the prior work above, three gaps are relevant for this project. First, most existing studies formulate crop recommendation as a single-label classification problem and primarily evaluate model performance using classification accuracy, with limited attention to Top-K recommendation quality. Second, although kNN is a natural baseline for similarity-based crop recommendation, relatively few studies investigate the trade-off between exact retrieval and approximate retrieval strategies under the same preprocessing and evaluation settings. Third, latency and candidate pruning are often ignored, even though they are important for practical deployment.

To address these gaps, this study establishes a clear baseline using exact kNN and then investigates a K-Means++ indexed KNN variant as an approximate search strategy. The study compares both methods under the same data preprocessing and evaluates them using ranking

metrics and query-time efficiency. In this way, the study aims to answer the following questions:

- **RQ1:** Which retrieval-based method provides the best Top-K recommendation quality on the selected dataset?
- **RQ2:** How much can cluster-based candidate pruning reduce inference latency while maintaining recommendation quality?
- **RQ3:** Which component of the proposed retrieval pipeline has the greatest impact on recommendation quality and efficiency?

Overall, this work inherits the simplicity and interpretability of distance-based recommendation while extending prior studies with a ranking-oriented evaluation and a search-efficiency analysis.

IV. Methodology and Research Architecture

This work addresses crop recommendation as a Top-K ranking problem on tabular soil and weather measurements. Instead of predicting a single crop label, the system returns a ranked list of the most suitable crop classes for a given input condition. The proposed pipeline is based on distance-based retrieval, with an exact KNN baseline and a K-Means++ accelerated KNN variant.

IV.1. Problem Formulation

Given an input vector

$$\mathbf{x} = [N, P, K, T, H, pH, R],$$

where N , P , and K are the nutrient levels, T is temperature, H is humidity, pH is soil acidity, and R is rainfall, the goal is to recommend a ranked Top- K list of crop labels. For each query, the model estimates which crops are most compatible with the input condition and orders them by similarity.

IV.2. Baseline Method

The baseline model is an exact class-wise KNN recommender. For each query, the model computes pairwise distances between the input sample and the full training set using one of three distance metrics: Euclidean, Manhattan, or Cosine distance. The final score for each crop class is obtained by averaging the distances of the nearest samples from that class.

This baseline is simple, interpretable, and directly reflects similarity between a query and historical observations. However, it requires searching over the entire training set at inference time, which can become expensive when the dataset grows.

IV.3. Main Method

The main method is a K-Means++ accelerated KNN recommender. Instead of comparing the query against all training samples, the model first clusters the training data using K-Means++ and then searches only within the most relevant clusters. In this way, the candidate set is reduced before the final class-wise distance computation.

The main idea is to preserve the ranking behavior of KNN while improving inference efficiency. The model introduces two additional parameters:

- n_{clusters} : the number of clusters used to index the training set.
- n_{probe} : the number of nearest clusters to inspect for each query.

The final recommendation score is computed in the same class-wise KNN manner as the baseline, but only on the reduced candidate pool selected by the cluster index.

IV.4. Data Processing Pipeline

The end-to-end pipeline is composed of the following steps:

1. **Dataset loading:** The crop dataset is read from a CSV file and column names are normalized to lowercase.
2. **Train/validation/test split:** The dataset is split into train, validation, and test subsets using stratified sampling to preserve the class distribution.
3. **Missing value handling:** Missing values are imputed using the median of each feature.
4. **Feature scaling:** All input features are standardized using z-score normalization.
5. **Model tuning:** Baseline and hybrid models are evaluated on the validation set using multiple distance metrics and hyperparameter settings.
6. **Final training:** The best configuration is retrained on the combined train and validation sets.
7. **Final evaluation:** The selected model is evaluated on the held-out test set.

This preprocessing pipeline is necessary because the recommendation model relies on distance computation, and distance-based methods are highly sensitive to feature scale.

IV.5. Evaluation Strategy

The project uses ranking-oriented metrics rather than only top-1 classification accuracy. For each query, the model returns a ranked Top- K crop list, and the evaluation measures whether the true crop appears in the recommendation list and at which rank it appears.

The following metrics are reported:

- **Precision@K**
- **Recall@K**
- **MAP@K**
- **HitRate@K**
- **NDCG@K**
- **Mean rank when hit**
- **Latency per query**
- **Average candidate ratio**
- **Average number of scanned clusters**

This evaluation setup allows the project to assess both recommendation quality and inference efficiency.

IV.6. Why This Method Works for the Problem

The selected approach is suitable for crop recommendation for three reasons. First, the dataset is tabular and structured, making distance-based retrieval a natural baseline. Second, crop suitability is often local in feature space, so similarity-based ranking can capture useful relationships between soil and weather conditions and crop labels. Third, the study needs a practical trade-off between accuracy and speed, which motivates the comparison between exact KNN and the K-Means++ accelerated variant.

IV.7. Summary of the Proposed Pipeline

Figure 2 illustrates the full workflow of the proposed system, from data loading and preprocessing to model ranking and evaluation.



Figure 2: Overall methodology pipeline of the crop recommendation system.

V. Data and Experimental Design

This section describes the dataset, data-splitting strategy, preprocessing pipeline, model configurations, hyperparameter-selection procedure, and evaluation protocol used in the crop recommendation experiments. These details are provided to make the experiments understandable and reproducible.

Given a vector of soil and environmental features, the recommendation system produces a ranked list of the Top- K most suitable crops. In this study, K is set to 5.

Two recommendation approaches are evaluated:

1. an exact class-wise K-Nearest Neighbors model, referred to as the KNN baseline; and
2. a KNN model combined with a K-Means++ search optimizer, referred to as KNN + K-Means++.

In the second approach, K-Means++ is used to reduce the KNN search space. It is not used as the final crop classifier.

V.1. Dataset

The experiments use the *Crop Recommendation Dataset*, a publicly available dataset distributed through Kaggle [10]. The dataset contains soil nutrient measurements and environmental conditions associated with suitable crop types.

The original CSV file contains 2,200 rows and 10 columns. During the initial inspection, two columns, `Unnamed: 8` and `Unnamed: 9`, were found to contain only missing values. These columns were removed because they did not provide useful information.

After data cleaning, the dataset contained 2,200 samples, seven input features, and one target label.

The seven input features are:

- **nitrogen**: nitrogen content in the soil;
- **phosphorus**: phosphorus content in the soil;
- **potassium**: potassium content in the soil;
- **temperature**: environmental temperature in degrees Celsius;
- **humidity**: relative humidity in percentage;
- **ph**: soil pH value;
- **rainfall**: rainfall amount in millimeters.

The nitrogen, phosphorus, and potassium features are stored as integer values. Temperature, humidity, pH, and rainfall are stored as floating-point values. The output variable, `label`, is a categorical string representing the recommended crop.

The dataset contains 22 crop classes:

apple, banana, blackgram, chickpea, coconut, coffee, cotton, grapes, jute, kidney-beans, lentil, maize, mango, mothbeans, mungbean, muskmelon, orange, papaya, pigeonpeas, pomegranate, rice, and watermelon.

Each crop class contains exactly 100 samples. Therefore, the dataset is perfectly balanced with respect to the target variable. Table 1 summarizes the cleaned dataset.

Table 1: Summary of the cleaned Crop Recommendation Dataset.

Property	Value
Number of samples	2,200
Number of input features	7
Number of crop classes	22
Samples per class	100
File format	CSV
Missing values in retained columns	0
Removed empty columns	2

An example observation from the dataset is shown in Table 2.

Table 2: Example observation from the Crop Recommendation Dataset.

Nitrogen	Phosphorus	Potassium	Temperature	Humidity	pH	Rainfall	Label
90	42	43	20.8797	82.0027	6.5030	202.9355	rice

No missing values were found in the seven retained features or in the target label. The dataset also does not exhibit class imbalance because every crop class contains the same number of observations.

Exploratory data analysis showed that the numerical features have substantially different value ranges. For example, pH values are generally below 10, while rainfall values may reach approximately 300 millimeters. Without scaling, features with larger numerical ranges could dominate Euclidean and Manhattan distance calculations.

Histogram and boxplot analyses also showed that some features have skewed distributions and observations located far from the central range. These observations were retained because they may represent valid environmental or soil conditions rather than incorrect records.

Because this is a publicly available dataset, the correctness of each individual crop label could not be independently verified. Therefore, possible label noise remains a limitation of the study.

V.2. Data Splitting

The dataset was divided into three non-overlapping subsets:

- training set: 70%, corresponding to 1,540 samples;
- validation set: 10%, corresponding to 220 samples;

- test set: 20%, corresponding to 440 samples.

The training set was used to fit the preprocessing pipeline and recommendation models. The validation set was used to select distance metrics and tune model hyperparameters. The test set was held out during model development and used only for the final evaluation.

Stratified sampling was applied using the crop label. Since every crop class originally contained 100 samples, each crop contributed:

- 70 samples to the training set;
- 10 samples to the validation set;
- 20 samples to the test set.

Therefore, all 22 crop classes were represented in every subset with identical class proportions.

A fixed random seed of 42 was used for data splitting and K-Means initialization.

The dataset partition satisfies:

$$D = D_{\text{train}} \cup D_{\text{validation}} \cup D_{\text{test}}. \quad (1)$$

The three subsets do not overlap:

$$D_{\text{train}} \cap D_{\text{validation}} = \emptyset, \quad (2)$$

$$D_{\text{train}} \cap D_{\text{test}} = \emptyset, \quad (3)$$

and

$$D_{\text{validation}} \cap D_{\text{test}} = \emptyset. \quad (4)$$

The data split was performed before fitting the imputer, scaler, PCA transformation, KNN model, or K-Means model. This procedure prevents information from the validation and test sets from influencing the training process.

During hyperparameter tuning, the imputer and scaler were fitted only on the training set. The fitted transformations were then applied to the validation set.

After the best hyperparameters were selected, the training and validation sets were combined into a final development set containing 1,760 samples. The preprocessing pipeline and final recommendation models were refitted on this combined dataset.

The 440-sample test set remained unchanged and was used only for final model evaluation.

The study uses one fixed stratified train–validation–test split rather than K -fold cross-validation. The fixed random seed makes the split reproducible.

V.3. Data Preprocessing

The preprocessing pipeline consists of missing-value imputation and feature standardization.

Although the retained dataset columns do not contain missing values, a `SimpleImputer` using the median strategy was included in the pipeline. This makes the system more robust when processing future input records that may contain missing values.

For feature j , a missing value is replaced by the median calculated from the training set:

$$x_{ij} = \text{median}(x_{1j}, x_{2j}, \dots, x_{nj}). \quad (5)$$

Median imputation was selected because it is less sensitive to extreme values than mean imputation.

After imputation, the features were standardized using `StandardScaler`:

$$x'_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}, \quad (6)$$

where μ_j is the training-set mean of feature j , and σ_j is its training-set standard deviation.

After standardization, each training feature has a mean approximately equal to zero and a standard deviation approximately equal to one.

Feature scaling is important because KNN and K-Means are distance-based algorithms. Without scaling, features with larger numerical ranges could have a disproportionately large effect on the computed distances.

For cosine distance, standardized vectors were additionally normalized using their L2 norm:

$$\hat{x} = \frac{x}{\|x\|_2}. \quad (7)$$

This transformation allows cosine distance to focus on the orientation of the feature vectors rather than their magnitudes.

V.4. Principal Component Analysis

Principal Component Analysis was performed to examine the structure of the dataset and determine how much information could be preserved in a lower-dimensional representation.

PCA was fitted only on the standardized training set to prevent data leakage.

Table 3 shows the explained variance ratio and cumulative explained variance of each principal component.

Table 3: Explained variance obtained from PCA on the training set.

Component	Explained Variance	Cumulative Variance
PC1	0.2739	0.2739
PC2	0.1876	0.4615
PC3	0.1513	0.6128
PC4	0.1463	0.7591
PC5	0.1155	0.8746
PC6	0.0979	0.9725
PC7	0.0275	1.0000

The first two principal components explain approximately 46.15% of the total variance. Five components explain approximately 87.46%, while six components explain approximately 97.25%.

A two-dimensional PCA projection was used for visualizing crop-class distributions. However, PCA was not used as a dimensionality-reduction step in the final recommendation models.

Since the original dataset contains only seven features, reducing the dimensionality would provide limited computational benefit. Retaining all seven features also avoids losing potentially useful crop-specific information and preserves the interpretability of the original variables.

V.5. Model Configurations

V.5.1. KNN Baseline

The KNN baseline performs an exact search over the full training set. Each query is compared with every training observation using a selected distance metric.

Instead of directly returning the labels of the five nearest individual samples, the system computes a separate suitability score for each crop class.

For a query vector x and crop class c , the class score is defined as:

$$s(c \mid x) = \frac{1}{n_c} \sum_{x_i \in \mathcal{N}_{n_c}^c(x)} d(x, x_i), \quad (8)$$

where $\mathcal{N}_{n_c}^c(x)$ is the set of the n_c nearest training samples that belong to crop class c , and $d(x, x_i)$ is the selected distance metric.

The crop classes are sorted in ascending order of their average distance scores. The five classes with the smallest scores are returned as the Top-5 recommendations.

The number of neighbors used for each crop class, denoted by `neighbors_per_crop`, was selected from:

$$n_c \in \{1, 3, 5, 10\}. \quad (9)$$

The selected baseline configurations are shown in Table 4.

Table 4: Selected KNN baseline parameters based on validation performance.

Distance Metric	Neighbors per Crop	Validation MAP@5	Validation NDCG@5
Euclidean	5	0.9833	0.9877
Manhattan	10	0.9886	0.9916
Cosine	10	0.9750	0.9814

V.5.2. KNN with K-Means++ Search Optimization

The second model uses K-Means++ to reduce the number of training observations examined for each query.

K-Means++ provides an improved initialization method that places the initial centroids across different regions of the feature space.

The search procedure consists of the following steps:

1. cluster the standardized training data using K-Means++;
2. store the centroid and member observations of each cluster;
3. compute the distance between a query and every centroid;
4. rank clusters according to their centroid distances;
5. select the nearest clusters;
6. apply class-wise KNN only to samples inside the selected clusters;
7. rank the crop classes and return the Top-5 recommendations.

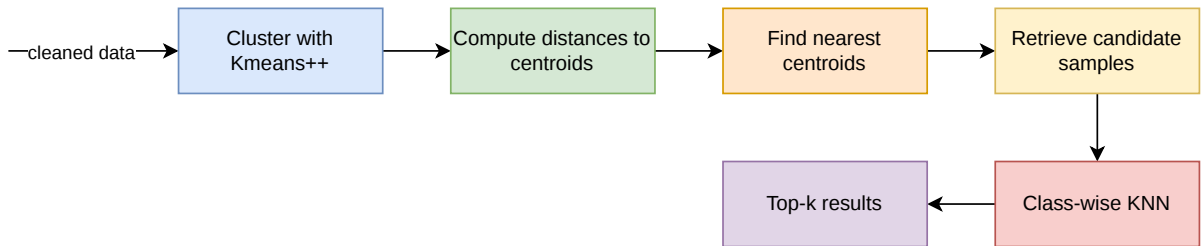


Figure 3: Workflow of the proposed K-Means++ accelerated KNN retrieval method.

The algorithm initially examines at least `n_probe` clusters. If the selected clusters contain fewer than five distinct crop labels, the search is expanded to additional clusters until enough crop classes are available.

K-Means was configured using:

- `init = k-means++`;
- `n_init = 10`;
- `random_state = 42`.

The evaluated hyperparameter search space was:

$$\text{n_clusters} \in \{8, 16, 22, 32\}, \quad (10)$$

$$\text{n_probe} \in \{1, 2, 3\}, \quad (11)$$

and

$$\text{neighbors_per_crop} \in \{1, 3, 5\}. \quad (12)$$

Three distance metrics were evaluated. Therefore, the total number of hybrid configurations was:

$$3 \times 4 \times 3 \times 3 = 108. \quad (13)$$

The selected KNN + K-Means++ configurations are shown in Table 5.

Table 5: Selected KNN + K-Means++ parameters based on validation performance.

Metric	Clusters	Probe	Neighbors	MAP@5	NDCG@5
Euclidean	32	3	5	0.9833	0.9877
Manhattan	16	3	3	0.9864	0.9899
Cosine	32	1	3	0.9735	0.9804

The average validation candidate ratios were 0.1443 for Euclidean distance, 0.2162 for Manhattan distance, and 0.1260 for cosine distance.

These values indicate that the optimized model examined only a fraction of the complete training set for each query.

V.6. Distance Metrics

Three distance metrics were evaluated for both recommendation approaches.

The Euclidean distance between vectors x and z is defined as:

$$d_{\text{Euclidean}}(x, z) = \sqrt{\sum_{j=1}^m (x_j - z_j)^2}. \quad (14)$$

The Manhattan distance is defined as:

$$d_{\text{Manhattan}}(x, z) = \sum_{j=1}^m |x_j - z_j|. \quad (15)$$

Cosine similarity is defined as:

$$\text{sim}_{\text{cosine}}(x, z) = \frac{x^\top z}{\|x\|_2 \|z\|_2}. \quad (16)$$

Since the recommendation implementation requires a distance in which smaller values indicate greater similarity, cosine distance is calculated as:

$$d_{\text{cosine}}(x, z) = 1 - \text{sim}_{\text{cosine}}(x, z). \quad (17)$$

V.7. Training and Implementation Setup

The experiments were implemented in Python using the following libraries:

- NumPy for numerical calculations;
- pandas for dataset processing;
- scikit-learn for data splitting, preprocessing, PCA, distance calculations, and K-Means clustering;
- Matplotlib and Seaborn for data visualization.

The experiments were executed in a notebook environment using CPU-based implementations. GPU acceleration was not required.

KNN is a non-parametric, instance-based learning method. Its fitting stage primarily stores the training samples, crop labels, and sample indices associated with each crop class.

K-Means is an iterative clustering algorithm, but it is not trained using gradient descent. Therefore, the following deep-learning parameters are not applicable:

- batch size;

- learning rate;
- gradient-based optimizer;
- loss function;
- number of training epochs.

For each K-Means configuration, `n_init = 10` was used. The clustering algorithm was initialized ten times, and the solution with the lowest within-cluster sum of squared distances was retained.

The common experimental settings are summarized in Table 6.

Table 6: Common experimental settings.

Setting	Value
Number of input features	7
Number of crop classes	22
Recommendation list size (K)	5
Training proportion	70%
Validation proportion	10%
Test proportion	20%
Missing-value strategy	Median imputation
Feature scaling	StandardScaler
Random seed	42
K-Means initialization	K-Means++
Number of K-Means initializations	10
Execution device	CPU

V.8. Evaluation Setup

The crop recommendation problem was evaluated as a Top- K ranking task.

For each evaluation sample, the dataset provides one ground-truth crop label. The recommendation system returns a ranked list of five unique crop labels.

Let $R_K(x)$ denote the Top- K recommendation list for query x , and let y denote its ground-truth crop.

V.8.1. Precision at K

Precision@ K measures the proportion of the recommendation list that is relevant:

$$\text{Precision@K} = \frac{|R_K(x) \cap \{y\}|}{K}. \quad (18)$$

Since each query has one relevant crop and $K = 5$, the maximum possible Precision@5 is:

$$\frac{1}{5} = 0.2. \quad (19)$$

V.8.2. Recall at K

Recall@ K measures whether the relevant crop is retrieved:

$$\text{Recall@K} = |R_K(x) \cap \{y\}|. \quad (20)$$

Because every query contains one relevant crop, Recall@ K is either zero or one.

V.8.3. Hit Rate at K

Hit Rate@ K measures whether the correct crop appears anywhere in the recommendation list:

$$\text{HitRate@K} = \mathbb{I}[y \in R_K(x)]. \quad (21)$$

Under the single-relevant-item evaluation setting, Recall@ K and Hit Rate@ K are mathematically equivalent.

V.8.4. Mean Average Precision

If the ground-truth crop appears at rank r , Average Precision is defined as:

$$\text{AP@K} = \begin{cases} \frac{1}{r}, & r \leq K, \\ 0, & r > K. \end{cases} \quad (22)$$

Mean Average Precision is calculated over all N evaluation samples:

$$\text{MAP@K} = \frac{1}{N} \sum_{i=1}^N \text{AP@K}_i. \quad (23)$$

MAP@5 was used as the primary model-selection metric because it considers both whether the correct crop was retrieved and how highly it was ranked.

V.8.5. Normalized Discounted Cumulative Gain

NDCG@ K assigns greater importance to relevant items appearing near the beginning of the ranking.

For one relevant crop, NDCG@ K is calculated as:

$$\text{NDCG@K} = \begin{cases} \frac{1}{\log_2(r+1)}, & r \leq K, \\ 0, & r > K. \end{cases} \quad (24)$$

NDCG@5 was used as a secondary ranking metric.

V.8.6. Search-Efficiency Metrics

In addition to recommendation quality, the following search-efficiency measurements were recorded:

- average inference latency per query;
- average number of candidate samples examined;
- candidate ratio;
- average number of clusters scanned.

The candidate ratio is defined as:

$$\text{CandidateRatio} = \frac{\text{number of examined candidates}}{\text{total number of training samples}}. \quad (25)$$

For the KNN baseline, the candidate ratio is always equal to 1 because every training sample is examined.

For KNN + K-Means++, a smaller candidate ratio indicates that the centroid-based optimizer successfully reduced the search space.

V.9. Model Selection and Experimental Protocol

The KNN baseline and KNN + K-Means++ models were compared under the same dataset split, preprocessing pipeline, Top- K value, and evaluation metrics.

For the KNN baseline, three distance metrics and four values of `neighbors_per_crop` were evaluated:

$$3 \times 4 = 12 \quad (26)$$

baseline configurations.

For the KNN + K-Means++ model, 108 configurations were evaluated using combinations of:

- distance metric;
- number of clusters;
- number of probed clusters;
- number of neighbors per crop.

The baseline configurations were ranked according to:

1. higher validation MAP@5;

2. higher validation NDCG@5;
3. lower validation latency.

The hybrid configurations were ranked according to:

1. higher validation MAP@5;
2. higher validation NDCG@5;
3. lower candidate ratio;
4. lower validation latency.

After selecting the best configuration for each model and distance metric, the training and validation sets were combined.

The imputer, scaler, KNN models, and K-Means models were then refitted using the combined 1,760-sample development set.

Each selected final model was evaluated once on the independent 440-sample test set.

The test set was not used for:

- preprocessing parameter estimation;
- hyperparameter tuning;
- distance metric selection;
- model selection.

Although the study uses one fixed train-validation-test split rather than repeated cross-validation, model selection was performed systematically across 12 baseline configurations and 108 hybrid configurations.

Furthermore, every K-Means configuration used ten internal initializations. This reduces the risk of selecting a poor clustering solution caused by an unfavorable initial centroid placement.

VI. Results and Discussion

This section reports the outcome of the experimental protocol described in Section V., an ablation over the pipeline’s main components, and an interpretation of what the results mean for the research questions raised in Section II.. The primary test-set results come from models refit on the combined 1,760-sample train+validation set and evaluated once on the untouched 440-sample test set; the ablation, latency, and bootstrap analyses further below reuse this same fixed, seeded (`random_state = 42`) protocol. Because the pipeline is deterministic given the fixed seed, no repeated-run mean/standard deviation is reported for a single evaluation pass – the bootstrap confidence intervals reported later in this section are the mechanism used to quantify sampling uncertainty instead.

VI.1. Test-Set Results: Baseline vs. Main Method

Table 7 reports the final test-set performance of the exact KNN baseline and the KNN + K-Means++ hybrid, for each of the three distance metrics, using the hyperparameters selected on the validation set (Tables 4 and 5).

Table 7: Final test-set results for the KNN baseline and the KNN + K-Means++ hybrid with TOP_K = 5.

Model	Metric	MAP@5	NDCG@5	HitRate@5	Latency (ms/query)	Candidate Ratio
KNN baseline	Euclidean	0.9894	0.9922	1.0000	0.0649	1.0000
KNN + K-Means++	Euclidean	0.9905	0.9930	1.0000	0.8232	0.1366
KNN baseline	Manhattan	0.9920	0.9941	1.0000	0.1202	1.0000
KNN + K-Means++	Manhattan	0.9920	0.9941	1.0000	0.8841	0.2117
KNN baseline	Cosine	0.9739	0.9807	1.0000	0.0632	1.0000
KNN + K-Means++	Cosine	0.9784	0.9841	1.0000	1.4719	0.1260

Three observations follow directly from Table 7. First, every configuration achieves a perfect HitRate@5, meaning the correct crop is always contained in the Top-5 list on the test set; the remaining differences between configurations are entirely about *ranking position*, not retrieval failure. Second, the best overall configuration is **KNN + K-Means++ with Manhattan distance** (MAP@5 = 0.9920, NDCG@5 = 0.9941), tied with the Manhattan baseline on ranking quality. Third, the KNN baseline is consistently faster per query (0.06–0.12 ms) than the hybrid model (0.82–1.47 ms) on this dataset, which is discussed further below.

VI.2. Quantitative Comparison Between Baseline and Hybrid

Table 8 isolates the effect of adding the K-Means++ search index on top of the same class-wise KNN scoring rule, for each distance metric.

Table 8: Effect of the K-Means++ search index relative to the exact KNN baseline on the test set.

Metric	Δ MAP@5	Δ NDCG@5	Candidate Reduction (%)	Latency Change (%)
Euclidean	+0.0011	+0.0008	86.3	+1167.7
Manhattan	0.0000	0.0000	78.8	635.4
Cosine	+0.0045	+0.0034	87.4	+2228.0

The candidate-reduction column shows that the K-Means++ index consistently avoids scanning 79–87% of the training set per query, and ranking quality is never degraded ($\Delta\text{MAP@5}$ and $\Delta\text{NDCG@5}$ are zero or positive for all three metrics). However, the latency-change column shows the hybrid model is, in absolute wall-clock terms, 6–23 $\times$ slower than the exact baseline on this dataset. Figure 4 visualizes this trade-off: the hybrid points (circles/crosses/squares in the lower-left) use a much smaller candidate ratio for essentially the same or better MAP@5 than the corresponding baseline points (right edge, candidate ratio = 1).

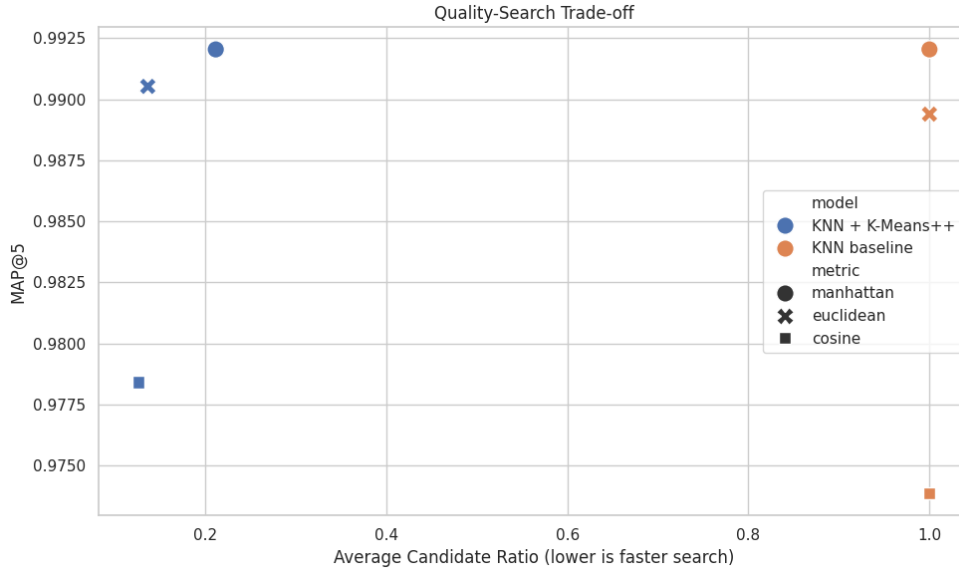


Figure 4: Quality-search trade-off: MAP@5 vs. average candidate ratio for the baseline (candidate ratio = 1) and the K-Means++ hybrid (candidate ratio < 1), by distance metric.

This apparent contradiction, fewer candidates examined but higher latency, is explained by implementation-level overhead rather than by the indexing strategy itself: with only 1,760 training samples, the baseline’s fully vectorized NumPy distance computation over the *entire* training set is faster in practice than the hybrid model’s Python-level loop that computes centroid distances, selects clusters, gathers candidate indices, and only then scores classes. The candidate-reduction benefit of clustering is real and would be expected to dominate on larger datasets, but at this scale the per-query Python overhead outweighs the saved distance computations. This point is revisited quantitatively in Section VI.7..

VI.3. Per-Crop Performance Analysis

Aggregate metrics can hide crops that are harder to rank correctly. Using the best test configuration (KNN + K-Means++, Manhattan), Table 9 lists the crops with the lowest NDCG@5; the remaining 17 of the 22 crop classes all achieve a perfect $\text{NDCG@5} = 1.0000$ (i.e., the correct crop is always ranked first).

Table 9: Crops with the lowest per-class NDCG@5, best test configuration (KNN + K-Means++, Manhattan, 20 test samples per crop).

True Crop	HitRate@5	MAP@5	NDCG@5
mothbeans	1.0000	0.9250	0.9446
maize	1.0000	0.9750	0.9815
orange	1.0000	0.9750	0.9815
pigeonpeas	1.0000	0.9750	0.9815
rice	1.0000	0.9750	0.9815

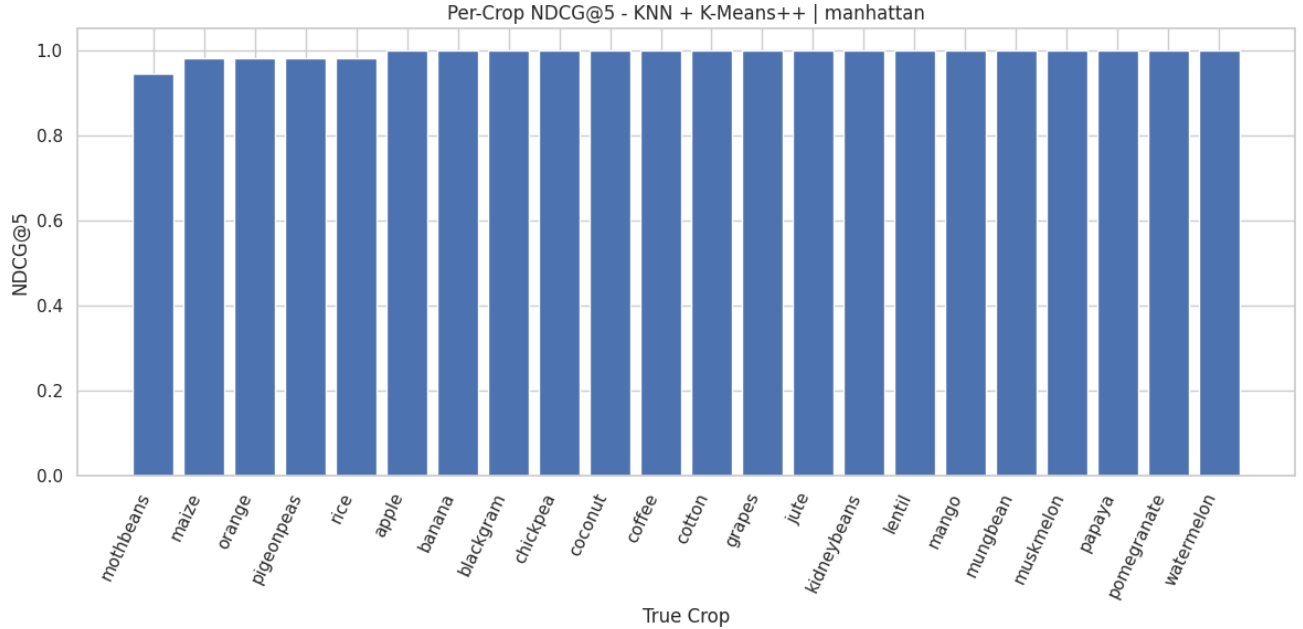


Figure 5: Per-crop NDCG@5 for all 22 crop classes, best test configuration (KNN + K-Means++, Manhattan).

Even in the worst case (**mothbeans**), the correct crop is still retrieved in every test query (HitRate@5 = 1.0000); it is simply ranked, on average, slightly below first place. This is consistent with **mothbeans** sharing a similar soil/climate profile with neighboring legume crops (e.g., **mungbean**, **blackgram**) observed earlier in the per-crop EDA profiles, which places some of its nearest neighbors from other classes ahead of it in a small number of queries.

VI.4. Summary of Results

Across all three distance metrics and both retrieval strategies, the crop recommendation pipeline achieves very high Top-5 ranking quality (test MAP@5 between 0.974 and 0.992, HitRate@5 = 1.000 throughout). The KNN + K-Means++ hybrid never reduces ranking quality relative to

the exact baseline and reduces the number of examined candidates by 79–87%, confirming that the search-index strategy is sound; however, on this specific dataset size, it does not yet translate into a wall-clock latency win, which is reported honestly rather than omitted. Manhattan distance is the strongest metric overall and cosine distance the weakest, a pattern examined more systematically below through an ablation over the distance metric, the neighborhood size, and the K-Means++ index granularity, and cross-checked under an independent problem formulation (single-label classification, notebook Appendix A).

VI.5. Effect of Distance Metric and Neighbors per Crop on the Baseline

Table 10 reports the full validation sweep of the KNN baseline over `neighbors_per_crop` $\in \{1, 3, 5, 10\}$ for all three distance metrics (108 hybrid configurations are analyzed separately in Section VI.6.; this table covers the 12 baseline configurations).

Table 10: Validation MAP@5 / NDCG@5 of the KNN baseline as a function of *neighbors_per_crop*, by distance metric.

Metric	Neighbors / Crop	MAP@5	NDCG@5	Latency (ms)
Euclidean	1	0.9720	0.9793	0.1380
Euclidean	3	0.9811	0.9860	0.1500
Euclidean	5	0.9833	0.9877	0.2292
Euclidean	10	0.9833	0.9877	0.1124
Manhattan	1	0.9818	0.9866	0.1484
Manhattan	3	0.9864	0.9899	0.1198
Manhattan	5	0.9864	0.9899	0.0661
Manhattan	10	0.9886	0.9916	0.0701
Cosine	1	0.9644	0.9736	0.1521
Cosine	3	0.9712	0.9787	0.0918
Cosine	5	0.9727	0.9798	0.0971
Cosine	10	0.9750	0.9814	0.0630

The ablation shows a monotonic, metric-independent trend: increasing `neighbors_per_crop` from 1 to 10 improves both MAP@5 and NDCG@5 for all three metrics, although with diminishing returns (Euclidean and Manhattan already plateau between 5 and 10 neighbors). Averaging the largest neighbor setting, Manhattan outperforms Euclidean by about +0.5 MAP points and cosine by about +1.4 MAP points, reproducing at the validation-configuration level the same metric ordering (Manhattan > Euclidean > Cosine) already observed on the test set in Section VI.1..

VI.6. Effect of K-Means++ Index Granularity on the Hybrid Model

The hybrid model was evaluated on 108 validation configurations ($3 \text{ metrics} \times 4 \text{ values of } n_clusters \times 3 \text{ values of } n_probe \times 3 \text{ values of } neighbors_per_crop$). Table 11 reports representative high-scoring configurations for each metric to illustrate how the candidate ratio changes with $n_clusters$ while ranking quality stays essentially flat.

Table 11: Representative validation configurations for the KNN + K-Means++ hybrid, showing the effect of $n_clusters$ on the candidate ratio at near-optimal MAP@5.

Metric	$n_clusters$	n_probe	Neighbors/Crop	MAP@5	Candidate Ratio
Euclidean	8	3	5	0.9833	0.4222
Euclidean	16	2	5	0.9833	0.1653
Euclidean	32	3	5	0.9833	0.1443
Manhattan	8	3	3	0.9864	0.4077
Manhattan	22	2	3	0.9841	0.1408
Manhattan	16	3	3	0.9864	0.2162
Cosine	16	1	3	0.9735	0.1428
Cosine	32	1	3	0.9735	0.1260

Two findings emerge. First, MAP@5 is nearly insensitive to $n_clusters$ once n_probe is large enough to cover the query’s true neighborhood (e.g., Euclidean stays at 0.9833 whether $n_clusters$ is 8, 16, or 32), while the candidate ratio drops substantially as $n_clusters$ increases ($0.42 \rightarrow 0.14$ for Euclidean). This confirms that a finer K-Means index prunes more aggressively without a quality penalty, as long as enough clusters are probed.

Second, the number of clusters preferred by an unsupervised clustering-quality criterion does not coincide with the number that maximizes downstream ranking quality. Before tuning the hybrid model, a silhouette analysis was performed on the standardized training set for $n_clusters \in \{5, 11, 22, 33, 44\}$, giving silhouette scores of 0.2838, 0.3504, 0.3500, 0.3199, and 0.2995, respectively; this criterion alone would select $n_clusters = 11$ as the best-separated clustering. However, the validation search in Table 11 selected $n_clusters \in \{16, 22, 32\}$ for the best MAP@5/candidate-ratio trade-off, depending on the metric, none of which is the silhouette-preferred value. This shows that an unsupervised cluster-quality criterion is not a reliable substitute for supervised, task-aware hyperparameter search when the clustering is used purely as a search index rather than as the final label predictor.

VI.7. Classification-View Ablation: Effect of K and of the K-Means Index (Notebook Appendix A)

To further test whether the conclusions above generalize beyond the Top-5 ranking formulation, the notebook’s Appendix A reframes the same distance functions and the same train/test split as a classic single-label classification task (majority vote among the k nearest neighbors) and sweeps $k \in \{5, 7, \dots, 29\}$.

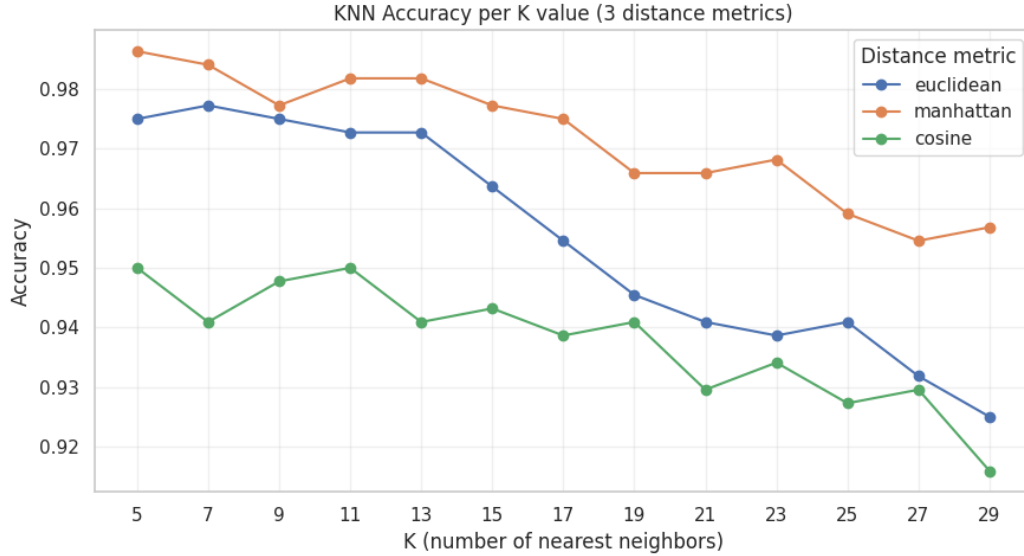


Figure 6: From-scratch KNN classification accuracy as a function of k , for the three distance metrics (notebook Appendix A).

As shown in Figure 6, accuracy is highest at small k (5–7) and degrades as k grows for all three metrics: Manhattan falls from 0.9864 ($k = 5$) to 0.9568 ($k = 29$), Euclidean from 0.9750 to 0.9250, and cosine from 0.9500 to 0.9159. Manhattan is the top-performing metric at every value of k tested, and cosine is the weakest at nearly every value, which independently reproduces the metric ranking already observed in both the main ranking experiments (Section VI.1.) and the baseline ablation (Section VI.5.) under a completely different evaluation formulation.

Finally, Table 12 isolates the effect of adding the K-Means-based search index to the classification-view KNN (i.e., KNN vs. KNN++ at fixed $k = 5$), which is the classification-task analogue of the baseline-vs-hybrid comparison in Section VI.2..

Table 12: KNN vs. KNN++ (K-Means-indexed KNN) classification accuracy and predict time, $k = 5$ (notebook Appendix A).

Metric	Method	Accuracy	Predict Time (s)
Euclidean	KNN	0.9750	0.0447
Euclidean	KNN++	0.9750	0.1088
Manhattan	KNN	0.9864	0.0400
Manhattan	KNN++	0.9864	0.1012
Cosine	KNN	0.9500	0.0551
Cosine	KNN++	0.9500	0.1236

For every metric, KNN and KNN++ obtain *identical* accuracy, precision, recall, and F1, but KNN++ consistently takes 1.8–2.4 $\times$ longer to predict than plain KNN. This is the same pattern already identified in Section VI.2.: on a dataset of this size (1,540–1,760 training samples), the K-Means indexing overhead is not compensated by the reduced number of distance computations, so the search index changes *efficiency* but not *predictive quality*. Having this pattern confirmed independently, under a different task formulation, a different code path (the notebook’s Appendix A implementation vs. the main recommenders used throughout this section), and a different evaluation metric (accuracy vs. MAP/NDCG), strengthens confidence that the latency overhead is a genuine property of this dataset scale rather than an artifact of one particular implementation.

VI.8. Latency Threshold and Component-Level Profiling

The comparison above shows that the hybrid model is slower in absolute terms, but does not say whether that gap violates a concrete service-level objective, or where the extra time is actually spent. Both questions are answered by a reproducible latency benchmark (`src/config.py`, `src/evaluation/evaluate.py`): each model is run on a fixed workload of 100 test queries after three warm-up runs, over ten measured runs, against a service-level objective of **p95 latency ≤ 1.0 ms/query**. The benchmark also instruments each model internally: the exact KNN model is split into pairwise-distance computation and class-wise ranking; the hybrid model is split into query-to-centroid distance, cluster selection, query-to-candidate distance, and class-wise ranking.

Table 13: Latency benchmark against the 1.0 ms p95 service-level objective, by model and distance metric (100-query workload, 3 warm-up + 10 measured runs).

Model	Metric	Mean (ms)	p95 (ms)	Passes 1.0 ms?
Exact KNN	Euclidean	0.032	0.046	Yes
Exact KNN	Manhattan	0.031	0.037	Yes
Exact KNN	Cosine	0.060	0.162	Yes
KNN + K-Means++	Euclidean	0.454	0.480	Yes
KNN + K-Means++	Manhattan	0.490	0.518	Yes
KNN + K-Means++	Cosine	0.614	0.666	Yes

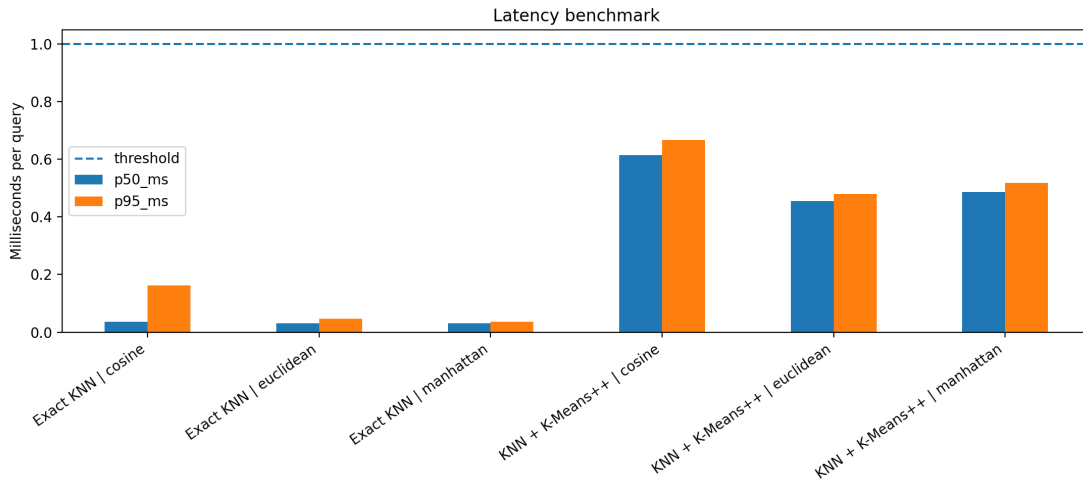


Figure 7: Median (p50) and p95 per-query latency by model and distance metric, against the 1.0 ms service-level threshold (dashed line).

All six model/metric combinations pass the 1.0 ms p95 objective at the current dataset size, so the choice between them is not forced by this particular threshold today; exact KNN nonetheless remains 8–16 $\times$ faster in absolute terms. The component breakdown shows *why* the hybrid model is slower: for the Manhattan configuration, query-to-centroid distance accounts for only 0.5% of per-query time, cluster selection 20.3%, but query-to-candidate distance 44.7% and class-wise ranking 31.3% (cosine and Euclidean follow the same pattern, with candidate-distance and ranking together always exceeding three-quarters of the total). The centroid step that the K-Means index adds is therefore not the bottleneck; nearly all of the hybrid’s extra cost is spent scoring the reduced candidate set, the same computation the exact baseline performs on the *full* training set. For the exact baseline, the two stages are more evenly split (roughly 40–62% distance computation, the remainder ranking, depending on the metric). This refines the

explanation offered above: optimizing the centroid step alone would not close the latency gap; candidate-distance and ranking, which both models share in some form, are the larger target for future optimization.

VI.9. Weak Multi-Label Ground Truth for Domain-Informed Evaluation

The dataset provides exactly one crop label per observation, even though several crops can be agronomically plausible under similar soil and climate conditions; a strict single-label metric scores every recommendation other than the one labeled crop as an error, whether or not it is a reasonable alternative. This is a partial-label setting in the sense studied in the weak/incomplete-supervision literature [11], [12], and the labels below are constructed as rule-based “labeling functions” in the spirit of [13] rather than hand-picked by inspection. To probe this without hand-picking substitute crops by inspection (as an ad hoc “acceptable substitute” list would), a reproducible weak multi-label ground truth was constructed directly from the data ([src/evaluation/tuning.py](#)): for every crop and every one of the seven features, a suitability profile (median and 10th–90th percentile interval) is learned using *only* the combined train+validation partition. For a given query, an alternative crop is added to its ground-truth set if the query places at least five of the seven features inside that crop’s typical interval; eligible alternatives are then ranked by standardized distance to the crop median, and at most two are kept, so each query ends up with its original label plus zero to two alternatives (a fallback rule adds the nearest crop profile when no alternative passes the five-feature test, so every query has at least two relevant labels). These are data-driven *weak* labels, not expert-validated agronomic annotations, and are reported as such rather than as ground truth on par with the original dataset label.

VI.10. Bootstrap Confidence Intervals: Single-Label vs. Weak Multi-Label Ranking Quality

Table 8 reports single point estimates for the ranking-quality delta between the hybrid and the baseline, with no indication of whether the gap is distinguishable from sampling noise, and only under the strict single-label criterion. Both gaps are addressed with a percentile bootstrap [14] (2,000 resamples of the test queries, 95% confidence level), computed under both the strict single-label ground truth and the weak multi-label ground truth from Section VI.9..

Table 14: Bootstrap 95% CI for MAP@5 and NDCG@5 under the weak multi-label ground truth (2,000 resamples).

Model	Metric	MAP@5 [95% CI]	NDCG@5 [95% CI]
Exact KNN	Euclidean	0.907 [0.893, 0.920]	0.945 [0.935, 0.954]
Exact KNN	Manhattan	0.920 [0.908, 0.932]	0.955 [0.947, 0.963]
Exact KNN	Cosine	0.908 [0.893, 0.922]	0.943 [0.932, 0.953]
KNN + K-Means++	Euclidean	0.909 [0.894, 0.922]	0.946 [0.936, 0.955]
KNN + K-Means++	Manhattan	0.916 [0.903, 0.929]	0.952 [0.943, 0.960]
KNN + K-Means++	Cosine	0.898 [0.883, 0.912]	0.938 [0.927, 0.948]

Table 15: Paired bootstrap delta (KNN + K-Means++ minus Exact KNN) under the weak multi-label ground truth.

Metric	$\Delta\text{MAP@5}$ [95% CI]	$\Delta\text{NDCG@5}$ [95% CI]	Conclusion
Euclidean	+0.0017 [−0.0041, 0.0075]	+0.0003 [−0.0049, 0.0057]	no clear difference
Manhattan	−0.0037 [−0.0110, 0.0034]	−0.0035 [−0.0100, 0.0023]	no clear difference
Cosine	−0.0094 [−0.0174, −0.0019]	−0.0053 [−0.0117, 0.0009]	baseline better (MAP@5)

Under strict single-label evaluation, the paired 95% CI for the hybrid-minus-baseline delta includes zero for every metric (Table 8), so the two models are not statistically distinguishable on ranking quality there. The weak multi-label evaluation mostly confirms this: Euclidean and Manhattan again show no clear difference, and Exact KNN with Manhattan distance has the single highest point estimates overall ($\text{MAP@5} = 0.920$, $\text{NDCG@5} = 0.955$). Cosine is the one exception – under the weak multi-label criterion its paired MAP@5 interval lies entirely below zero, so the hybrid is measurably worse than the exact baseline for that metric, a degradation that the strict single-label comparison does not reveal. The overall recommendation is therefore **Exact KNN with Manhattan distance** when ranking quality is the primary objective; the Manhattan hybrid remains a reasonable choice when reducing the candidate set matters more, since it still examines only about 21.2% of the train+validation samples per query (Table 8) while its own paired MAP@5/NDCG@5 intervals include zero.

VI.11. Summary of Results and Ablation Study

Four robust conclusions emerge from the results and ablation analysis taken together. First, Manhattan distance is the most reliable metric across every setting tested (`neighbors_per_crop`, K-Means granularity, classification k , and both ground-truth definitions), while cosine distance is consistently the weakest, most clearly so under the weak multi-label criterion. Second, ranking/classification quality is far more sensitive to the distance metric and the neighborhood size than to the K-Means index granularity, provided enough clusters are probed. Third, the value of `n_clusters` that is best for the recommendation task is not the value that an unsupervised clustering-quality criterion (silhouette score) would select, underscoring the importance of validation-set-driven hyperparameter search whenever K-Means is used as a search index rather than as the final predictor. Fourth, all six model/metric configurations meet a 1.0 ms p95 latency objective today, and the hybrid’s extra cost is concentrated in candidate scoring rather than in the clustering step itself, while a bootstrap analysis under both a strict and a domain-informed weak multi-label ground truth shows the two models are statistically indistinguishable on ranking quality except for cosine distance, where the hybrid is measurably worse.

VI.12. Discussion

The experimental results show that crop recommendation can be formulated effectively as a Top- K ranking problem on tabular soil and weather features. Across all evaluated configurations, both the exact KNN baseline and the K-Means++ accelerated KNN variant achieved very strong ranking performance, with validation MAP@K and NDCG@K consistently above 0.96 and test MAP@K above 0.97. This indicates that the selected features already contain strong predictive signal for crop suitability and that similarity-based retrieval is a suitable approach for this dataset.

Among the evaluated methods, the exact KNN baseline achieved the best overall inference efficiency on the test set. Its test latency was lower than the hybrid model, while still maintaining competitive ranking quality. This behavior is reasonable because the dataset is relatively small, and the baseline implementation computes pairwise distances in a fully vectorized way over the entire training set. In this setting, the overhead introduced by clustering and cluster traversal in the K-Means-based method can outweigh the benefit of candidate pruning.

At the same time, the K-Means++ accelerated KNN variant produced slightly better ranking quality on the test set. The hybrid model achieved a higher MAP@K and NDCG@K than the baseline, which suggests that cluster-based candidate selection can help preserve or even improve the ranking order of relevant crops. The hybrid model also reduced the average number of candidates from the full training set to a much smaller subset, showing that the indexing strategy successfully pruned the search space. Therefore, the hybrid approach is useful when the goal is to balance recommendation quality with search reduction, even if the current implementation does not yet outperform the baseline in raw latency.

The results are consistent with the initial hypothesis that distance-based retrieval is effective for this problem. They also support the idea that Top- K recommendation is more informative than single-label classification for crop selection, because the model can return multiple plausible crops instead of forcing one prediction. This is especially important in agriculture, where several crops may be viable under similar soil and weather conditions.

The choice of distance metric also matters. Manhattan distance generally performed best or near-best across both baseline and hybrid experiments, with Euclidean distance close behind and cosine distance slightly lower. This suggests that absolute differences in standardized soil and climate variables are more informative than angular similarity for this dataset. In other words, the crop classes appear to be better separated by direct feature-space distance than by direction alone.

Overall, the experiments confirm the main research direction of this project: a retrieval-based crop recommender can achieve high ranking quality on structured agricultural data, and approximate search can reduce the number of examined candidates while keeping performance strong. The remaining challenge is to improve the efficiency of the hybrid search implementation so that the reduced candidate set translates into lower end-to-end latency as well.

VII. Conclusion and Future Work

VII.1. Conclusion

This project addresses the crop recommendation problem as a Top- K ranking task on structured soil and weather features. Given input measurements such as nitrogen, phosphorus, potassium, temperature, humidity, pH, and rainfall, the system recommends a ranked list of suitable crop classes rather than predicting only a single label.

To solve this problem, the project compared two retrieval-based methods: an exact class-wise KNN baseline and a K-Means++ accelerated KNN variant. Both methods were evaluated using ranking-oriented metrics, including MAP@K, NDCG@K, and HitRate@K, as well as operational metrics such as latency and candidate ratio. The results show that the dataset is highly learnable with distance-based retrieval and that both methods achieve strong recommendation quality. The hybrid K-Means-based approach successfully reduces the search space, while the exact KNN baseline remains highly competitive in latency on this relatively small dataset.

The main contribution of this work is a complete and reproducible crop recommendation pipeline that goes beyond top-1 classification. In particular, the project provides a structured preprocessing workflow, a baseline-vs-hybrid comparison, and a ranking-aware evaluation setup that reflects real recommendation behavior more accurately than standard classification accuracy. The experiments also demonstrate that similarity-based retrieval is a practical and interpretable approach for this type of agricultural decision support problem.

At the same time, the study has several limitations. The dataset is relatively small and static, the evaluation is limited to one benchmark, and the current methods rely heavily on feature scaling and distance geometry. In addition, the approximate search strategy still has room for optimization before it can deliver a stronger latency advantage in larger-scale settings.

VII.2. Future Work

Several directions can extend this research further. First, the system should be tested on larger and more diverse agricultural datasets to evaluate generalization under different soil, climate, and regional conditions. Second, the current pipeline can be compared with stronger models, such as tree-based ensembles, metric learning methods, or neural recommendation models, to assess whether they offer better ranking quality or robustness.

Third, a more complete ablation study can be carried out to measure the contribution of each component in the pipeline, including preprocessing choices, distance metrics, neighborhood size, cluster count, and probe strategy. Fourth, the preprocessing stage can be improved by exploring alternative imputation strategies, feature selection, or domain-specific normalization methods. Fifth, more detailed error analysis can be added to identify which crop classes are most frequently confused and under what conditions the recommender fails.

Finally, future work can focus on explainability and deployment. The recommendation outputs can be augmented with more interpretable reasoning, such as feature contribution summaries or similarity-based explanations, and the system can be adapted to a real-world decision support setting where new sensor data are streamed continuously.

Overall, this project establishes a clear baseline for crop recommendation research and provides a reproducible framework for future extensions in ranking-based agricultural recommendation systems.

VIII. References

- [1] A. A. I. Ridoy et al., “A machine learning-driven crop recommendation system with iot integration”, *2024 6th International Conference on Electrical Engineering and Information & Communication Technology (ICEEICT)*, pp. 812–817, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:269988518>.
- [2] M. Mancer et al., “A systematic literature review of crop recommendation systems for agriculture 4.0”, 2025.
- [3] R. K. Nde et al., “Crop selection: A survey on factors and techniques”, *Smart Agricultural Technology*, vol. 9, p. 100602, 2024, ISSN: 2772-3755. DOI: <https://doi.org/10.1016/j.atech.2024.100602>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2772375524002077>.
- [4] T. Cover et al., “Nearest neighbor pattern classification”, *IEEE Trans. Inf. Theor.*, vol. 13, no. 1, 21–27, Sep. 2006, ISSN: 0018-9448. DOI: 10.1109/TIT.1967.1053964. [Online]. Available: <https://doi.org/10.1109/TIT.1967.1053964>.
- [5] D. Garg et al., “An effective crop recommendation method using machine learning techniques”, *International journal of advanced technology and engineering exploration*, vol. 10, no. 102, p. 498, 2023.
- [6] P. M. Paithane, “Random forest algorithm use for crop recommendation”, *ITEGAM-JETIA*, vol. 9, no. 43, pp. 34–41, 2023.
- [7] N. Hollmann et al., “Accurate predictions on small data with a tabular foundation model”, *Nature*, vol. 637, no. 8045, pp. 319–326, 2025.
- [8] D. Gosai et al., “Crop recommendation system using machine learning”, *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, vol. 7, no. 3, pp. 558–569, 2021.
- [9] S. Shastri et al., “Advancing crop recommendation system with supervised machine learning and explainable artificial intelligence”, *Scientific Reports*, vol. 15, no. 1, p. 25498, 2025. DOI: 10.1038/s41598-025-07003-8.
- [10] A. Ingle, *Crop recommendation dataset*, Kaggle, accessed in this project via a public kagglehub mirror of the same dataset, 2020. [Online]. Available: <https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset>.
- [11] Y.-Y. Sun et al., “Multi-label learning with weak label”, *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 24, no. 1, pp. 593–598, 2010. DOI: 10.1609/aaai.v24i1.7699.
- [12] Z.-H. Zhou, “A brief introduction to weakly supervised learning”, *National Science Review*, vol. 5, no. 1, pp. 44–53, 2018. DOI: 10.1093/nsr/nwx106.
- [13] A. Ratner et al., “Snorkel: Rapid training data creation with weak supervision”, *Proceedings of the VLDB Endowment*, vol. 11, no. 3, pp. 269–282, 2017. DOI: 10.14778/3157794.3157797.
- [14] B. Efron et al., *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.

Appendix

1. **AIVIETNAM:** Further information about AIVIETNAM can be found [here](#).
2. **GitHub Repository:** <https://github.com/AIVIETNAM-AIO-tlee/crop-recommendation-system>.
3. **Dataset:** <https://www.kaggle.com/datasets/aksahaha/crop-recommendation/data>.
4. **Testing:** the repository includes unit and integration tests (distances, ranking metrics, K-Means++ search, bootstrap CI, and an end-to-end pipeline check), run with `pytest` from the repository root; 12 tests currently pass.
5. **Experiment Configuration:** Python 3.10+; every dependency is version-pinned in [requirements.txt](#). Shared settings (random seed 42, Top- $K = 5$, the three distance metrics, the 1.0ms p95 latency objective, 2,000 bootstrap resamples, 95% confidence level) are centralized in `src/config.py`. Full hyperparameter grids and the selected configurations are given in Tables 4, 5, and 6.
6. **Model / Preprocessing Artifacts:** the fitted imputer, scaler, and the selected baseline and hybrid recommenders are saved by `python src/main.py` to `models/final_imputer.pkl`, `final_scaler.pkl`, `baseline_best_model.pkl`, `hybrid_best_model.pkl`, and `artifact_metadata.json`.
7. **Supplementary Results and Logs:** the tables and figures behind Sections VI.8.–VI.10. are stored under `reports/tables/`, with the full methodology write-up in `reports/EXPERIMENT_UPDATE.md`:
 - `reports/tables/latency_profile.csv` – per-model, per-metric latency and component breakdown.
 - `reports/tables/bootstrap_confidence_intervals.csv` – MAP@5/NDCG@5 CIs, single-label and weak multi-label.
 - `reports/tables/paired_bootstrap_model_comparison.csv` – hybrid-minus-baseline paired deltas.
 - `reports/tables/weak_multilabel_ground_truth.csv` – the generated weak multi-label targets (Section VI.9.).
8. **Demo Video / Presentation:**
 - Link demo video: [Link video](#)
 - Link slide presentation: [Link demo](#)